



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Project Hardware and Software for Big Data mod A

M.s DATA SCIENCE

Prof. Giancarlo Sperli

Mina Roostapourdeilamany (D03000132)

February 2025

[Introduction]

Sentiment analysis is a crucial task in natural language processing (NLP) that aims to determine the sentiment expressed in a given piece of text. This project focuses on classifying tweets as either positive or negative using machine learning techniques. The dataset used for this classification task is the Sentiment140 dataset, which contains 1,600,000 tweets labeled with sentiment polarity. The objective is to evaluate different classification models and analyze their performance using key metrics such as accuracy, precision, recall, F1-score, and the confusion matrix.

[Task Definition]

The main goal of this project is to build a sentiment classification model capable of distinguishing between positive and negative tweets. The dataset contains tweets labeled as follows:

- ❖ 0: Negative sentiment
- ❖ 4: Positive sentiment

We preprocess the dataset, extract relevant features, and apply machine learning classifiers to predict the sentiment of unseen tweets. The performance of the models is evaluated using standard classification metrics.

[Data Description]

The dataset used in this project consists of 1,600,000 tweets labeled as either positive or negative. It was originally collected using the Twitter API and contains the following fields:

- **Sentiment:** The target variable (0 = negative, 4 = positive)
- **IDs:** A unique identifier for each tweet
- **Date:** The timestamp of the tweet
- **User:** The author of the tweet
- **Text:** The tweet's content

Since our focus is on sentiment classification, we only use the **text** and **sentiment** fields, discarding other metadata.

In the following sections, we will go through the code step by step, explaining the whole code.

[Code]

Before starting our code we Install these 3 Libraries in compute section on our cluster:

- wordcloud
- Nltk
- Pillow==11.0.0

Explanation of Imported Libraries

In this project, various Python libraries are used to handle data, preprocess text, visualize insights, and train machine learning models for sentiment analysis. Below is an explanation of each library and its purpose:

1. General Utilities

These libraries help with fundamental data operations and performance optimization.

re – Used for regular expressions, helping clean and preprocess text by removing unwanted characters like URLs, mentions, and special symbols.

time – Helps in measuring execution time and optimizing performance.

pickle – Used to serialize and save trained models or processed data, allowing for reuse without retraining.

numpy – A fundamental library for numerical computing, used to handle arrays and perform mathematical operations efficiently.

pandas – Essential for data manipulation and analysis, providing powerful data structures like DataFrames to handle large datasets.

2. Data Visualization

These libraries help in understanding data distributions and model performance through visualizations.

matplotlib.pyplot – A popular plotting library used to create visualizations such as histograms, bar charts, and line plots.

seaborn – Built on top of Matplotlib, it provides more aesthetically pleasing statistical visualizations, such as heatmaps for correlation analysis.

3. Word Cloud Visualization

To better understand the most frequently used words in tweets, word clouds are generated.

wordcloud.WordCloud – Creates visual representations of word frequency, helping to identify common words in positive and negative tweets.

PIL – Part of the Pillow library, it is used for image processing, including generating and customizing word clouds.

pip – A package manager for Python libraries, used to install missing dependencies.

4. Natural Language Processing (NLP)

Text preprocessing is crucial for sentiment analysis, and the Natural Language Toolkit (NLTK) provides essential tools.

nltk – A leading NLP library that offers tokenization, lemmatization, stopword removal, and other linguistic tools.

nltk.stem.WordNetLemmatizer – Reduces words to their base forms (e.g., “running” → “run”), improving text standardization and reducing vocabulary size.

nltk.download('wordnet') – Ensures that the WordNet corpus is available for lemmatization.

5. Machine Learning Models

These classifiers are used to train models that can distinguish between positive and negative sentiments.

sklearn.svm.LinearSVC – A Support Vector Machine (SVM) classifier that works well for text classification by finding the best hyperplane to separate sentiment classes.

sklearn.naive_bayes.BernoulliNB – A Naïve Bayes classifier that assumes independence among features, often performing well in text classification.

sklearn.linear_model.LogisticRegression – A commonly used linear model that predicts sentiment probabilities based on text features.

6. Model Training and Evaluation

To build and assess machine learning models, we use the following Scikit-learn utilities:

sklearn.model_selection.train_test_split – Splits the dataset into training and testing subsets to evaluate model performance.

sklearn.feature_extraction.text.TfidfVectorizer – Converts text into numerical feature vectors using Term Frequency-Inverse Document Frequency (TF-IDF), which helps in understanding word importance.

sklearn.metrics.confusion_matrix – Creates a matrix that shows correct and incorrect predictions, helping in evaluating model performance.

sklearn.metrics.classification_report – Provides a summary of classification metrics such as accuracy, precision, recall, and F1-score.

Loading and Preparing the Dataset

```
# Define the column names
DATASET_COLUMNS = ["sentiment", "ids", "date", "flag", "user", "text"]

# Load the CSV file with Spark and convert it to a Pandas DataFrame
dataset = spark.read.format('csv') \
    .options(header='false', delimiter=',', inferSchema='true') \
    .load('/FileStore/tables/training_1600000_processed_noemoticon.csv') \
    .toPandas()

# Manually assign column names
dataset.columns = DATASET_COLUMNS

# Show the first few rows of the Pandas DataFrame
dataset.head()
```

This part of the code is responsible for loading the Sentiment140 dataset using Apache Spark and converting it into a Pandas DataFrame for further processing. Since the dataset does not have predefined column names, they are manually assigned.

These columns represent:

- sentiment – The target label (0 = negative, 4 = positive).
- ids – A unique identifier for each tweet.
- date – The timestamp of when the tweet was posted.

- flag – A field indicating the query used (if applicable).
- user – The username of the person who posted the tweet.
- text – The actual content of the tweet.

Loading the CSV File Using Apache Spark:

- `spark.read.format('csv')` – Uses Apache Spark to load the dataset in CSV format.
- `.options(header='false')` – Indicates that the dataset does not have a header row.
- `.options(delimiter=',')` – Specifies that the values are comma-separated.
- `.options(inferSchema='true')` – Automatically detects and assigns data types to each column.
- `.load('/FileStore/tables/...')` – Specifies the path where the dataset is stored.
- `.toPandas()` – Converts the Spark DataFrame into a Pandas DataFrame

Why Use Apache Spark?

- ✓ Spark is used to handle large datasets efficiently.
- ✓ It supports distributed computing, making it faster than standard Pandas operations for big data.

	sentiment	ids	date	flag	user	text
0	0	1467810369	Mon Apr 06 22:19:45 PDT 2009	NO_QUERY	_TheSpecialOne_	@switchfoot http://twitpic.com/2y1zl - Awww, t...
1	0	1467810672	Mon Apr 06 22:19:49 PDT 2009	NO_QUERY	scotthamilton	is upset that he can't update his Facebook by ...
2	0	1467810917	Mon Apr 06 22:19:53 PDT 2009	NO_QUERY	mattycus	@Kenichan I dived many times for the ball. Man...
3	0	1467811184	Mon Apr 06 22:19:57 PDT 2009	NO_QUERY	ElleCTF	my whole body feels itchy and like its on fire
4	0	1467811193	Mon Apr 06 22:19:57 PDT 2009	NO_QUERY	Karoli	@nationwideclass no, it's not behaving at all...

Data Cleaning and Preparation

This step ensures that the dataset is clean, understandable, and ready for text preprocessing and model training

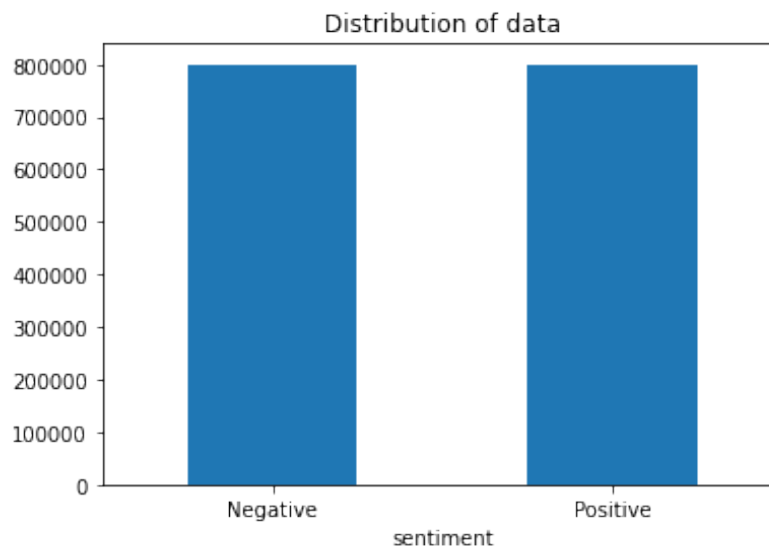
Removing Unnecessary Columns: Keeps only the sentiment and text columns since other columns are not needed for sentiment classification.

Replacing Sentiment Values for Clarity: Originally, the dataset labels positive sentiment as 4 and negative as 0. This replaces 4 with 1, making it clearer (0 = Negative, 1 = Positive).

Visualizing Data Distribution: Groups tweets by sentiment and plots a bar chart to show the dataset's balance between positive and negative tweets.

Storing Data for Further Processing: Converts the text and sentiment columns into separate lists for easier processing in later stages.

The bellow graph demonstrates the frequency bar charts of negative and positive twits.



Text Preprocessing Function

In this section we first define a dictionary and a list. And then create the function that does prepares the texts for recognizing sentiment.

Emoji Dictionary

steps help clean and normalize the text, making it more suitable for sentiment analysis.

```
emojis = {'😄': 'smile', '😊': 'smile', '😉': 'wink', '😞': 'sad', '😕': 'confused', ...}
```

This dictionary maps commonly used emojis and emoticons to their corresponding word meanings. It helps convert emojis into readable words, improving text consistency for sentiment analysis.

Stopword List

```
stopwordlist = ['a', 'an', 'is', 'are', 'he', 'she', 'i', 'you', 'here', 'there', 'in', 'on', ...]
```

Stopwords are common words (e.g., "the," "is," "in") that do not add much meaning to sentiment classification. Removing them reduces noise and improves the efficiency of machine learning models

Function PreProcess(text)

```
def preprocess(textdata):
```

This function **prepares tweets** for sentiment analysis by applying multiple text-processing techniques:

1. **Initialize Tools** : Uses WordNetLemmatizer to convert words to their base form (e.g., "running" → "run")
2. **Define Regex Pattern** : Detects and replaces URLs, usernames, special characters, and repeated letters and also converts "happyyyy" → "happy" to standardize words.
3. **Iterate Through Tweets and Clean Text** : Converts text to lowercase for uniformity and replaces URLs with "URL" to remove unnecessary web links.
4. **Replace Emojis with Their Meanings** : Converts emojis like :) into meaningful words like "EMOJIsmile".
5. **Replace User Mentions and Special Characters** : Replaces @username with "USER" to remove personal identifiers and removes special characters except letters and numbers.
6. **Lemmatization and Stopword Removal** :
Short words ("a", "i") are ignored and words are lemmatized to their base form for consistency.

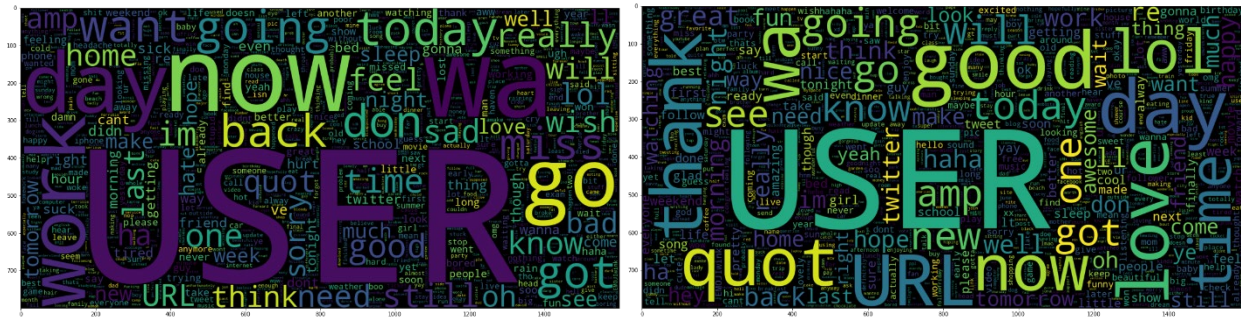
This function ensures that tweets are standardized, simplified, and free from unnecessary information, improving sentiment classification accuracy, and finally, it returns the outputs as a **list of cleaned tweets**, ready for vectorization and model training.

Generating a Word Cloud

In this section we visualize the most common words in negative sentiment tweets using a word cloud then it joins all words into a single string for processing.

The purpose is to Generate a word cloud from the most frequent words and displays the word cloud using `plt.imshow(wc)`

It assumes that the first 800,000 tweets in `processedtext` are negative and the others are positive.



Splitting Data and Vectorizing Text for Model Training

This section prepares the dataset for machine learning by splitting it into training/testing sets and converting text into numerical features.

1. Splitting Data into Training and Testing Sets

The model is trained on 95% of the data and tested on the remaining 5% to **evaluate performance**.

- **train_test_split()** splits the dataset into:
 - `X_train` & `X_test` → Processed tweets (input features).
 - `y_train` & `y_test` → Sentiment labels (0 = negative, 1 = positive).
- **test_size=0.05** → Reserves 5% of the data for testing and 95% for training.
- **random_state=0** → Ensures reproducibility by setting a fixed seed

2. Initializing and Fitting a TF-IDF Vectorizer

TF-IDF gives more importance to rare but meaningful words (e.g., "fantastic") rather than common ones (e.g., "the").

Capturing bigrams helps retain phrase-level meaning (e.g., "not good" vs. "good").

- `TfidfVectorizer( )` → converts text into a numerical matrix based on Term Frequency-Inverse Document Frequency (TF-IDF).
- `ngram_range = (1,2)` → Captures single words (unigrams) and word pairs (bigrams) to preserve context.
- `max_features = 500000` → Limits the vocabulary to the 500,000 most important words to reduce dimensionality.

3. Checking the Vocabulary Size

Prints the total number of unique words/features extracted from the dataset. It ensures the vocabulary is large enough to capture meaningful patterns but not too large to cause overfitting

Model Evaluation Function

In this section we define a function that evaluates a trained classification model using various performance metrics and a confusion matrix visualization. This function is crucial for assessing model performance and identifying weaknesses in classification.

```
def model_Evaluate(model):
```

- ✓ **Predicts sentiments on test data** → It allows comparison of predicted vs. actual values for evaluation.
- ✓ **Displays classification metrics (Precision, Recall, F1-score, Accuracy)** → It provides a detailed performance breakdown of the model and generates a classification report containing:
 - ✳ **Precision** (how many predicted positives are correct).
 - ✳ **Recall** (how many actual positives were correctly predicted).
 - ✳ **F1-score** (harmonic mean of precision & recall).
 - ✳ **Accuracy** (overall correctness of predictions)
- ✓ **Creates a confusion matrix to analyze errors** → Confusion Matrix shows the count of TN,FP,FN,TP and it helps analyze where the model makes errors (e.g., many false positives?)
 - ✳ **True Negatives (TN)** → Correctly predicted negative tweets.

- ✱ False Positives (FP) → Incorrectly predicted positive tweets.
- ✱ False Negatives (FN) → Incorrectly predicted negative tweets.
- ✱ True Positives (TP) → Correctly predicted positive tweets.

- ✓ **Visualizes results using Seaborn heatmap** → Plotting the confusion matrix by using the Seaborn's heatmap to create a visually clear confusion matrix. (Labels x-axis (Predicted) and y-axis (Actual) to show class comparisons.)
This work will provides intuitive visualization of how well the model classifies sentiments.

Model Evaluation and Comparison

We trained and evaluated three different models:

1. Bernoulli Naïve Bayes (BNB)
2. Support Vector Classifier (SVC)
3. Logistic Regression (LR)

Each model was tested on 80,000 samples, and we analyzed performance using Precision, Recall, F1-score, and Accuracy

1. Bernoulli Naïve Bayes (BNB)

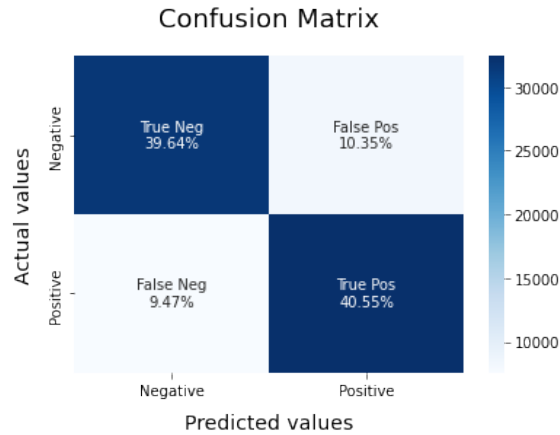
	precision	recall	f1-score	support
0	0.81	0.79	0.80	39989
1	0.80	0.81	0.80	40011
accuracy			0.80	80000
macro avg	0.80	0.80	0.80	80000
weighted avg	0.80	0.80	0.80	80000

Strengths:

- Fast training speed, suitable for large datasets.
- Works well with binary text classification problems.

Weaknesses:

- Lower recall (0.79 for class 0) indicates that some negative tweets were misclassified.
- Slightly lower accuracy (80%) compared to other models



2. Support Vector Classifier (SVC)

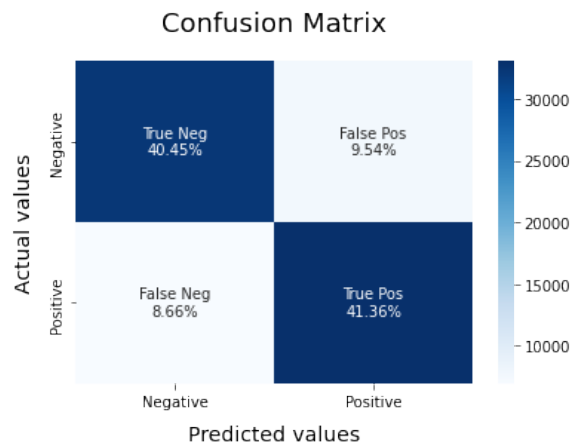
	precision	recall	f1-score	support
0	0.82	0.81	0.82	39989
1	0.81	0.83	0.82	40011
accuracy			0.82	80000
macro avg	0.82	0.82	0.82	80000
weighted avg	0.82	0.82	0.82	80000

Strengths:

- Higher accuracy (82%) than Naive Bayes.
- Balanced recall and precision, reducing misclassifications.

Weaknesses:

- More computationally expensive than BNB.
- May struggle with highly imbalanced datasets



3. Logistic Regression (LR)

	precision	recall	f1-score	support
0	0.83	0.82	0.83	39989
1	0.82	0.84	0.83	40011
accuracy			0.83	80000
macro avg			0.83	80000
weighted avg			0.83	80000

Strengths:

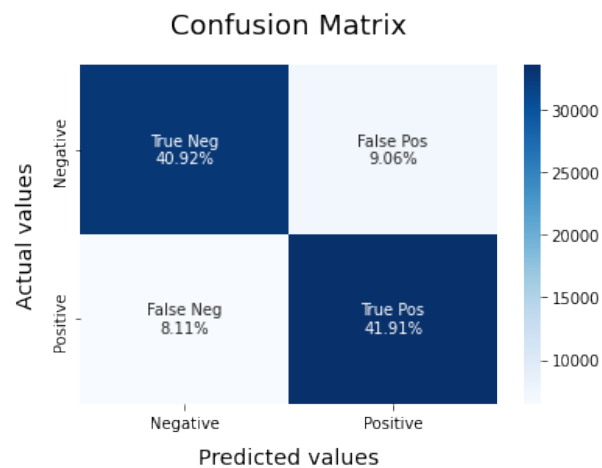
Best overall performance (83% accuracy).

Higher recall (0.84 for positive tweets) means fewer positive tweets were misclassified.

Weaknesses:

Slightly slower training compared to BNB.

Can struggle with non-linear data patterns



Comparison

Model	Precision	Recall	F1-score	Accuracy
BernoulliNB	0.80	0.80	0.80	80%
Support Vector (SVC)	0.82	0.82	0.82	82%
Logistic Regression	0.83	0.83	0.83	83%

- ★ **Logistic Regression performed the best overall with highest accuracy (83%)** and balanced precision/recall.
- ★ **SVC was slightly better than BNB**, achieving **82% accuracy**, making it a good alternative.
- ★ **BNB was the fastest model but had the lowest accuracy (80%)**.

Final Choice:

If performance is the priority → **Logistic Regression**

If speed is important → **Bernoulli Naïve Bayes**

For a balance of both → **Support Vector Classifier**

Model Saving, Loading, and Prediction

This final section of the code is focused on:

1. **Saving trained models** (Vectorizer, Logistic Regression, and Naive Bayes).
2. **Loading saved models** for future predictions.
3. **Making sentiment predictions** on new text input.

Running the Prediction

Four example sentences are classified as Positive or Negative sentiment using the Logistic Regression model and the prediction results are printed as a DataFrame.

```
      text sentiment
0  I am a good girl  Positive
1  That house is old  Negative
2    I Hate mornings  Negative
3    I like sunrise   Positive
```